

HW 4.3: Reproducible Data Analysis

Riya Dsouza

2026-04-15

Busiest Airports Analysis

Table 1: Passenger counts for busiest airports

Airport	2025	2024	2023	2022	2021
Beijing Daxing International Airport	49441029	NA	NA	25051012	NA
Dubai International Airport	92331506	86994365	66069981	29110609	25836771
Frankfurt Airport	61564957	59355389	48918482	24812849	18768601
Hartsfield–Jackson Atlanta International Airport	108067766	104653451	93699630	75704760	42918685
Heathrow Airport	83884572	79183364	61614508	NA	22111469
Los Angeles International Airport	76588028	75050875	65924298	48007284	28779527

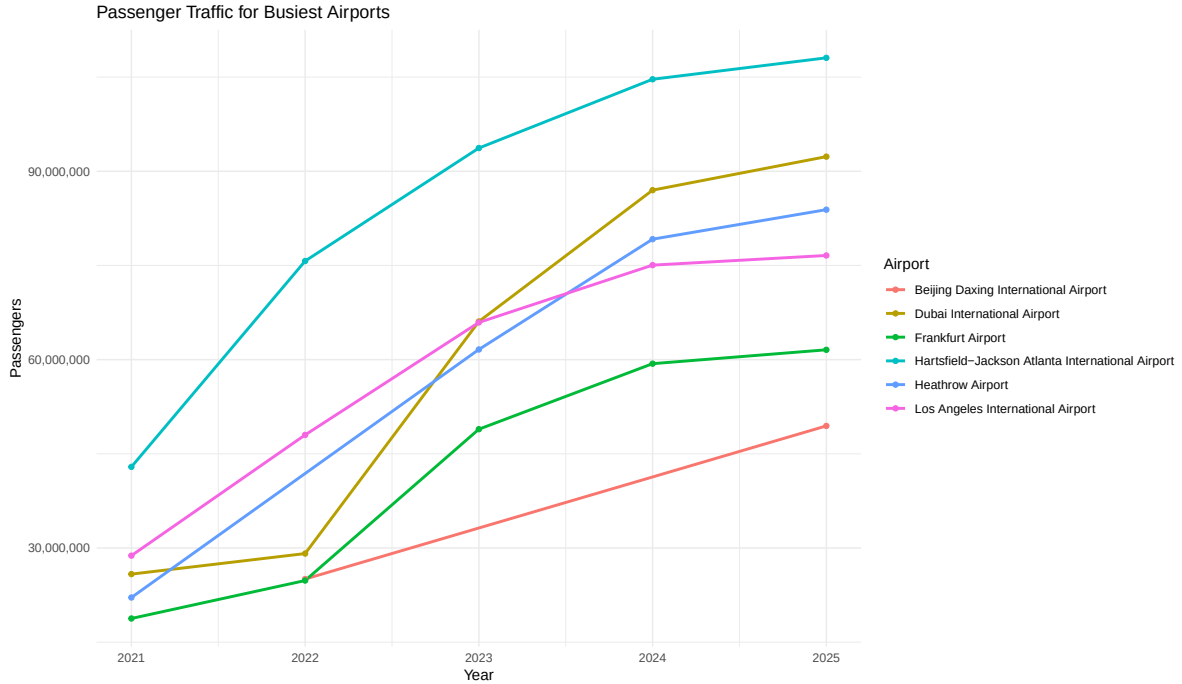


Figure 1: Passenger traffic trends for selected busiest airports.

The table and graph show how passenger traffic changes from 2020 to 2025 for several airports. Most airports increase over time, mostly because travel increased after COVID-19. Atlanta International Airport has the highest passenger numbers and stays consistent each year. Dubai and Heathrow also show strong growth. Some airports, like Beijing and Frankfurt are missing certain years or have lower values, which shows they were not always in the top rankings. The graph helps show trends over time, while the table gives exact numbers for comparison.

Monte Carlo Numerical Integration

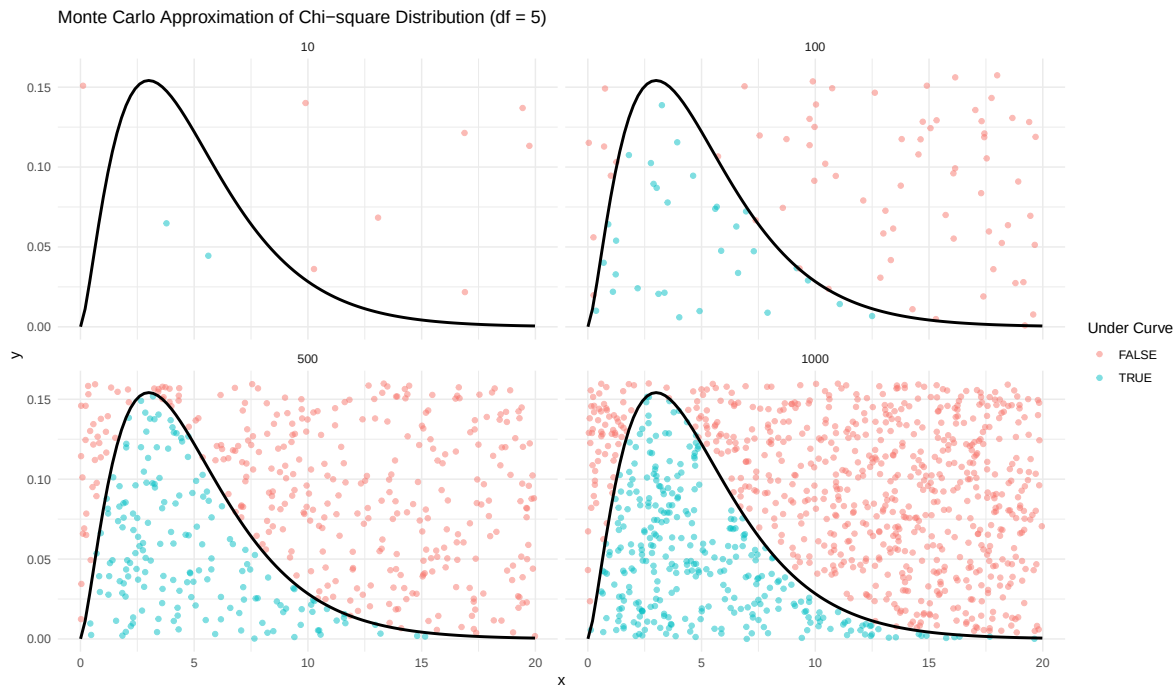


Figure 2: Monte Carlo approximation of the Chi-square distribution (df = 5).

Monte Carlo integration estimates the area under a curve by generating random points and checking whether they fall below the curve. The proportion of points under the curve is used to estimate the integral. In this case, the function is a Chi-square probability density function, so the exact value of the integral is 1. The small multiple shows that as the number of points increases, the estimate becomes more accurate. With fewer points, the estimate is rough, but with more points the values stabilize. This supports that the estimate is approaching 1.

Planning and Prompting GenAI Tools

Plan

Goal:

The goal is to clean and organize the calcium dataset so that changes in ulnar calcium measurements can be clearly analyzed over time. Then, I will create a visualization that shows how calcium levels differ across time and between the two treatment groups.

Verbs and Nouns Needed:

Nouns: computer with r installed on it, dataset, variables, calcium levels, groups, plot, graph

Verbs: load, clean, filter, select, check, inspect, tidy, create, adjust, make

Steps:

1. Make a plan.
2. Load the calcium dataset into R.
3. Inspect the dataset to understand the variables.
4. Check for missing or incorrect values and clean the data.
5. Ensure that variables are in the correct format.
6. Select the relevant variables.
7. Make sure that the dataset is in tidy format, where each row represents a single observation.
8. Create a visualization using ggplot2.
9. Check the plot to ensure it clearly shows differences between treatment groups over time.
10. Adjust labels, scales, or formatting to improve clarity and interpretation of the graph.

GenAI Output (Plan-Informed Response)

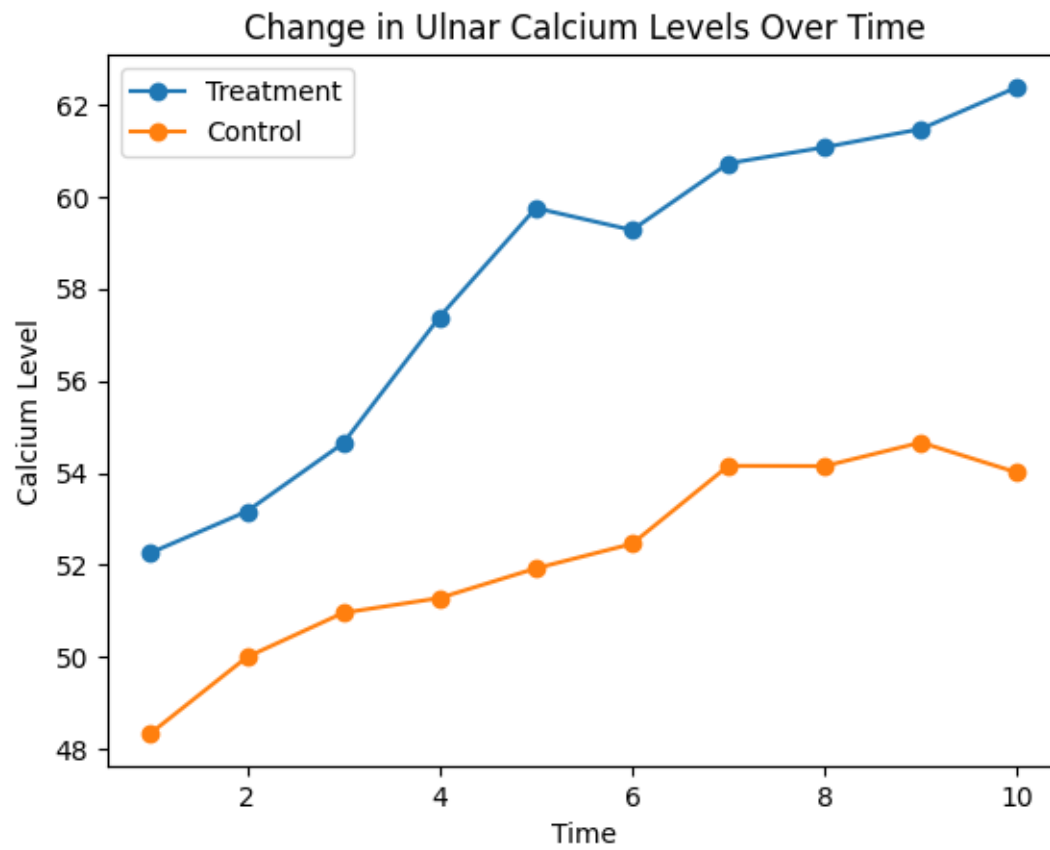


Figure 3: Output generated by ChatGPT using a generic prompt for the calcium dataset.

GenAI Output (Generic Prompt)

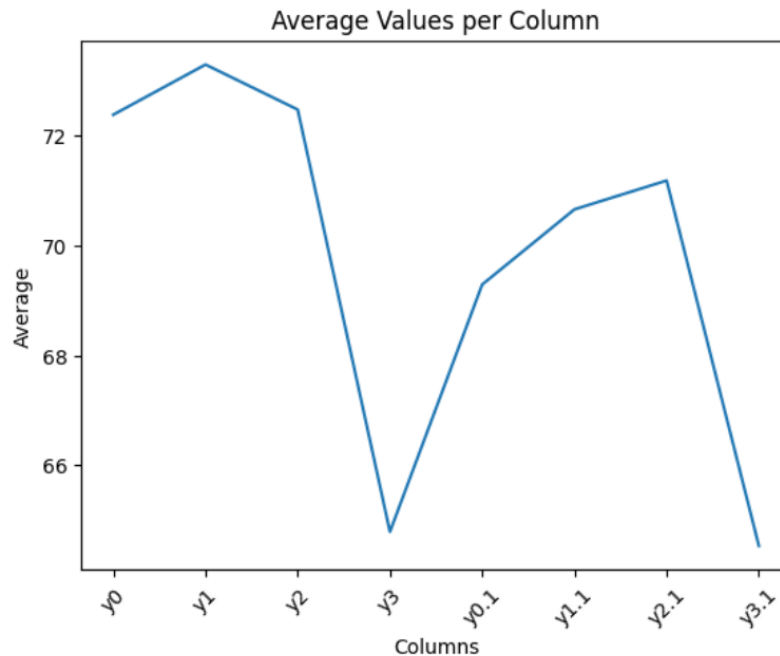


Figure 4: Output generated by ChatGPT, and provided in the assignment.

Narrative text showing the response of ChatGPT, which was provided in the assignment: I created a visualization showing the average value of each column in your dataset. What this shows: Each point represents the mean (average) of one column (y0, y1, y2, y3, etc.). You can quickly compare which variables tend to have higher or lower values overall. For example, some columns cluster around ~70–73, while others drop closer to ~65.

If you want something more insightful, I can also create:

Boxplots (to show spread and outliers)

Time series plots (if rows represent time/order)

Correlation heatmaps

Side-by-side comparisons (e.g., y0 vs y0.1)

Just tell me

Prompt: What is your version information?

You're chatting with ChatGPT based on the GPT-5.3 architecture.

Key details:

Model: GPT-5.3

Knowledge cutoff: August 2025

Current date (system): March 31, 2026

If you need more technical details (capabilities, differences vs GPT-4/5, etc.), just let me know

(This chat took place on Tuesday, March 31st, 2026.)

Comparison of Plan-Based vs Generic Prompting

The plan-informed response, the generic response, and my own work show clear differences in both the visualization and the overall understanding of the problem. The generic response creates a plot of average values by column, but it does not show the structure of the dataset or the goal of comparing treatment groups over time, making it uninformative for this context. In contrast, the plan-informed response produces a line graph that shows the correct structure and style the graph should have, but does not reflect the actual dataset. My own work focuses on organizing the data into a tidy format and visualizing changes over time, which helps show patterns more clearly. This comparison shows that having a clear plan leads to more accurate and useful outputs from genAI tools, while generic prompts can miss important points of the data. Overall, this shows that genAI tools are most effective when given structured guidance, but the strongest and most effective results still come from careful and independent work.

Reflection

Throughout this semester, I have learned how to approach data analysis in a more structured and intentional way. One of the most important skills I developed is learning how to plan before coding. For example, in each assignment, we first create a plan that includes a clear goal, the verbs and nouns needed, and a list of steps to take. This has helped me break down complex problems into smaller, manageable parts and made my code more organized and effective. I have also become more comfortable working with tidy data and using tools like ggplot2 to create clear graphs. Earlier in the semester, I focused mainly on just producing the output, but now I think more about whether the visualization actually communicates the intended message. Additionally, I learned how to use Quarto to combine code, results, and explanations into one reproducible document. This will improve how I present my work and it will make it easier to follow. Overall, this course has helped me build strong skills not only in coding, but also in planning, organizing, and clearly communicating data analysis.

Appendix 1: GenAI Usage

Usage 1: Planning and Prompting Section

Tool: ChatGPT

Version: GPT-5.3

Date: April 14, 2026

Prompts used: Using this information, provide me with a visualization of the following: The goal is to clean and organize the calcium dataset so that changes in ulnar calcium measurements can be clearly analyzed over time. Then, I will create a visualization that shows how calcium levels differ across time and between the two treatment groups. Verbs and Nouns Needed: Nouns: computer with r installed on it, dataset, variables, calcium levels, groups, plot, graph Verbs: load, clean, filter, select, check, inspect, tidy, create, adjust, make Steps: 1. Make a plan. 2. Load the calcium dataset into R. 3. Inspect the dataset to understand the variables. 4. Check for missing or incorrect values and clean the data if necessary. 5. Ensure that variables are in the correct format. 6. Select the relevant variables. 7. Make sure that the dataset is in tidy format, where each row represents a single observation. 8. Create a visualization using ggplot2. 9. Check the plot to ensure it clearly shows differences between treatment groups over time. 10. Adjust labels, scales, or formatting to improve clarity and interpretation of the graph. **Exact text response generated by the tool:** I generated a sample visualization for you based on what your are asking for.

Download it here: [Download the plot](#)

Important (don't ignore this)

This is a mock visualization, not your actual data.

That means:

It shows the correct structure and style your graph should have but it does NOT reflect your actual dataset.

Appendix 2: Code Appendix

```
library(tidyverse)
library(rvest)
library(kableExtra)

# read data
url <- "https://en.wikipedia.org/wiki/List_of_busiest_airports_by_passenger_traffic"
page <- read_html(url)
tables <- page |> html_table(fill = TRUE)

# build dataset
t2025 <- tables[[2]] |> mutate(year = 2025)
t2024 <- tables[[3]] |> mutate(year = 2024)
t2023 <- tables[[4]] |> mutate(year = 2023)
t2022 <- tables[[5]] |> mutate(year = 2022)
t2021 <- tables[[6]] |> mutate(year = 2021)

data <- bind_rows(t2025, t2024, t2023, t2022, t2021)

# clean
data <- data |>
  mutate(
    passengers = Totalpassengers |>
      gsub("\\[..*?\\]", "", x = _) |>
      gsub(",", "", x = _) |>
      as.numeric()
  )

tidy_data <- data |>
  select(Airport, year, passengers) |>
  mutate(Airport = str_trim(Airport))

# select airports
selected_airports <- c(
  "Hartsfield-Jackson Atlanta International Airport",
  "Frankfurt Airport",
  "Beijing Daxing International Airport",
  "Dubai International Airport",
  "Heathrow Airport",
  "Los Angeles International Airport"
)
```

```

# table
airport_table <- tidy_data |>
  filter(Airport %in% selected_airports) |>
  pivot_wider(names_from = year, values_from = passengers) |>
  arrange(Airport)

knitr::kable(
  airport_table,
  digits = 0,
  booktabs = TRUE,
  caption = "Passenger counts for busiest airports"
) |>
  kableExtra::kable_styling(
    latex_options = c("hold_position", "scale_down")
  ) |>
  kableExtra::column_spec(1, width = "6cm")
#plot
plot_data <- tidy_data |>
  filter(Airport %in% selected_airports)

ggplot(plot_data, aes(x = year, y = passengers, color = Airport)) +
  geom_line(linewidth = 1) +
  geom_point() +
  labs(
    title = "Passenger Traffic for Busiest Airports",
    x = "Year",
    y = "Passengers",
    color = "Airport"
  ) +
  scale_y_continuous(labels = scales::comma) +
  theme_minimal()
library(tidyverse)

# chi-square function
f <- function(x) {
  dchisq(x, df = 5)
}

# resolutions
n_values <- c(10, 100, 500, 1000)

results <- map_dfr(n_values, function(n) {

```

```

# get points
x <- runif(n, 0, 20)
y <- runif(n, 0, 0.16)

# classify
under <- y <= f(x)

tibble(
  x = x,
  y = y,
  under = under,
  n = n
)
})
# plot
ggplot(results, aes(x = x, y = y, color = under)) +
  geom_point(alpha = 0.5) +
  stat_function(fun = f, color = "black", linewidth = 1) +
  facet_wrap(~ n) +
  labs(
    title = "Monte Carlo Approximation of Chi-square Distribution (df = 5)",
    x = "x",
    y = "y",
    color = "Under Curve"
  ) +
  theme_minimal()

```